Networking Programming

Objectives

- Overview Networking Basic
- Overview Client-Server Model
- Explain about URL, URN and URI
- Explain about WebRequest and WebResponse class
- Explain about HttpClient class
- Explain about Domain Name System (DNS)
- Overview TCP Services: TcpListener, TcpClient and Socket class
- Demo WebRequest and HttpClient with .NET application
- Demo TcpListener and TcpClient with .NET application
- Explain and demo about UDP service with .NET application

Why Should We Study This Lecture?

- Nowadays, distributed applications are popular. People need large applications, running based on a computer network (local area networks-LANs- or wide area network-WAN), including many sites working concurrently. Do you want to create such applications?

- How do we develop network applications by .NET?

Networking Basics

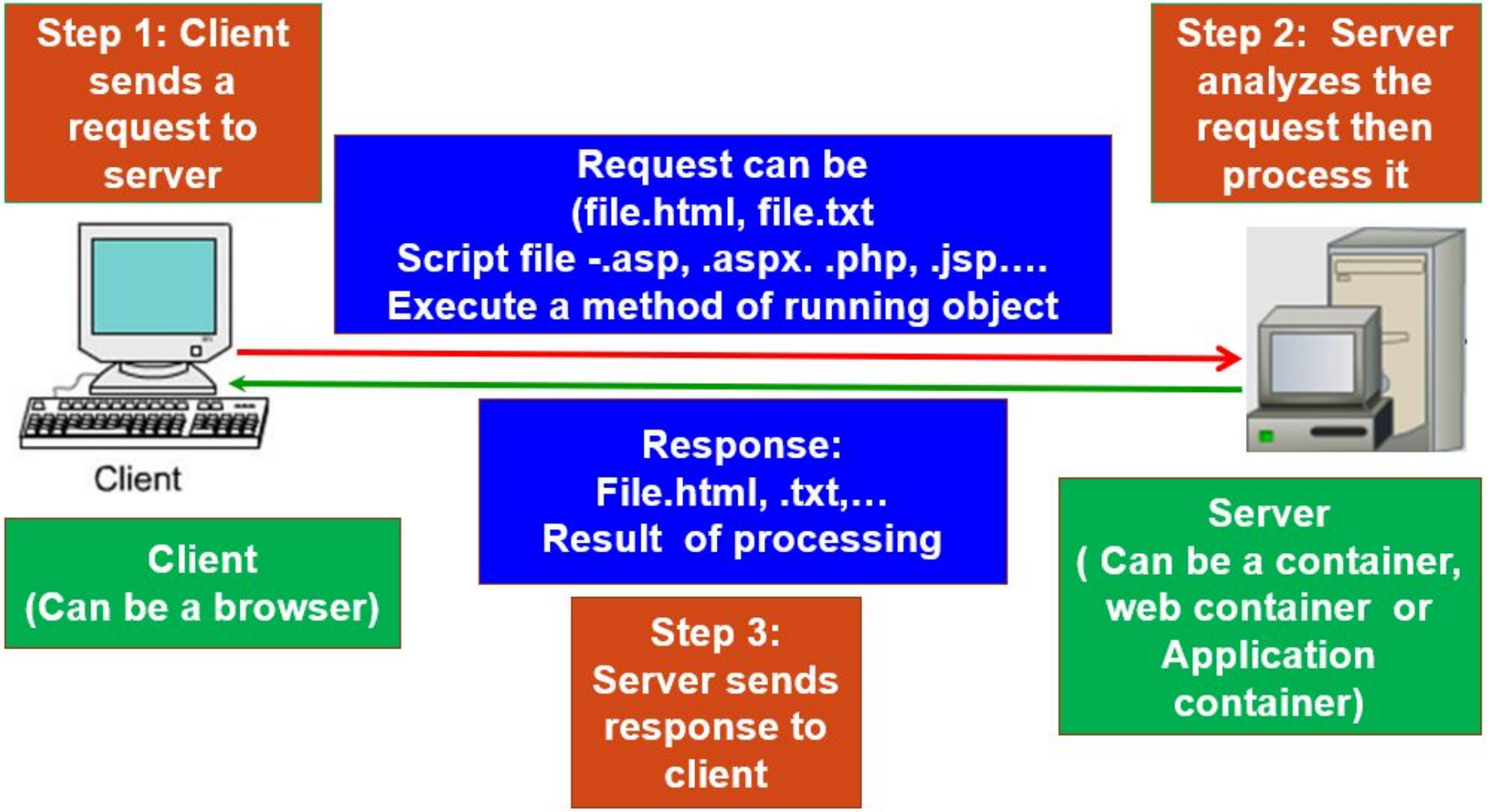
Some Definitions Related to Networking

- Platform: Hardware + Operating system
- Client: an application running in a computer (such as browser) can receive data from another (server)
- Server: an application running in a computer (such as IIS- Windows Internet Information Service, Kestrel, Nginx) can supply data to others (clients)
- IP address (Internet Protocol): unsigned integer helps identifying a network element(computer, router,…)
- IPv4: 4-byte IP address, such as 192.143.5.1
- IPv6: 16-byte IP address
- Port: unsigned 2-byte integer helps operating system differentiating a network communicating process

Some Definitions Related to Networking

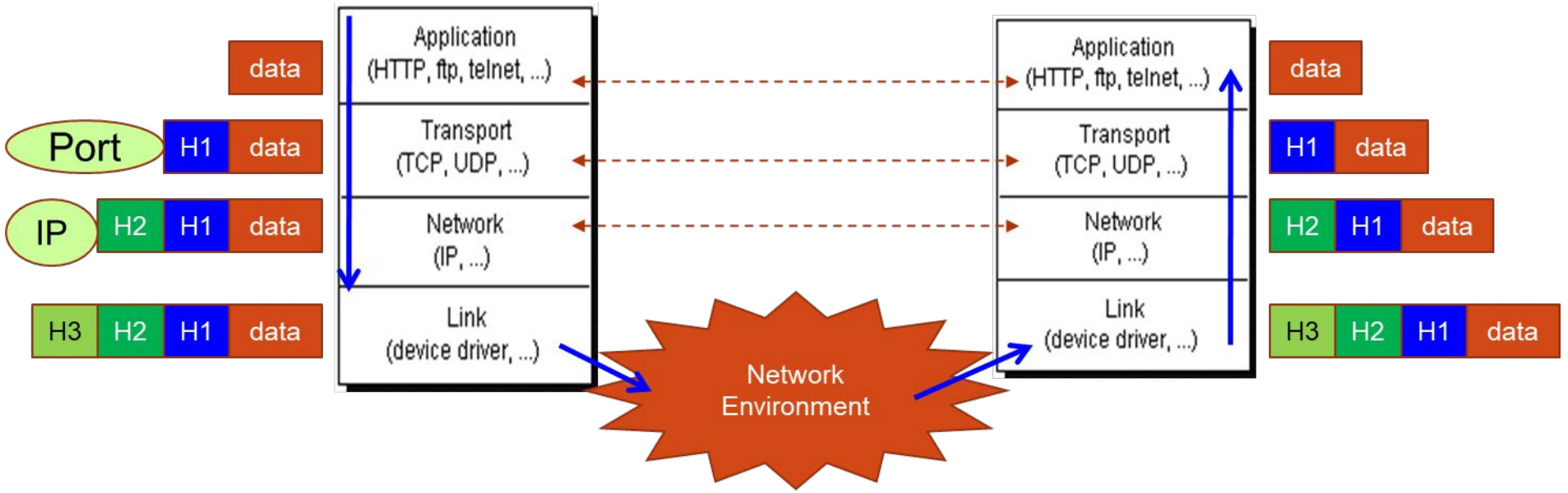
- Protocol: Rules for packaging data of a network communication because client and server can be working in different platform. Two common basic protocols are TCP and UDP
- TCP: (Transmission Control Protocol) is a connection-based protocol (only one connecting line only) that provides a reliable flow of data between two computers based on the acknowledge mechanism
- UDP: (User Datagram Protocol) is a protocol that sends independent packets of data, called datagrams, from one computer to another with no guarantees about arrival (many connecting lines can be used, acknowledge mechanism is not used). Many firewalls and routers have been configured not to allow UDP packets. Ask our system administrator if UDP is permitted

Client-Server Model



Client-Server Model

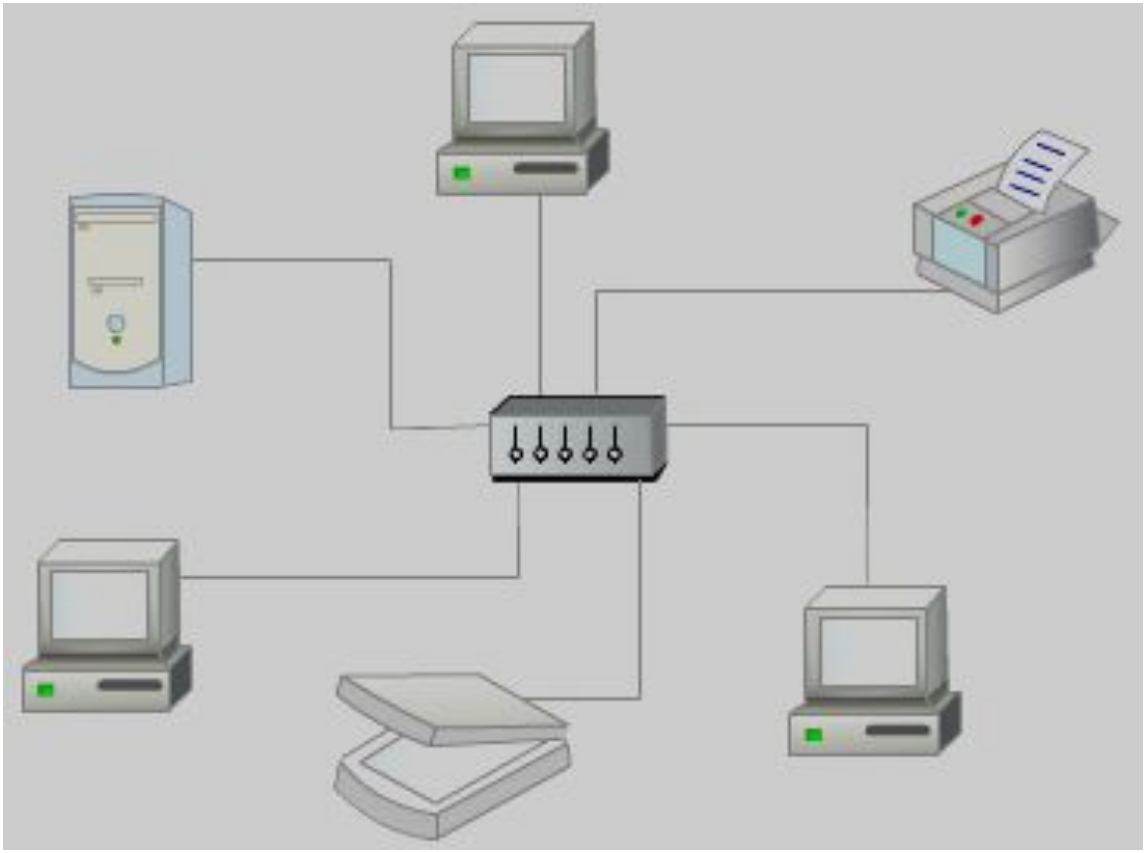
- Computers running on the Internet communicate to each other



A package is attached an appropriate header (H-identifiable data) when it is transferred to each layer. A layer is an applications or a function library of network managing system

Client-Server Model

- How to distinguish a computer in a network?



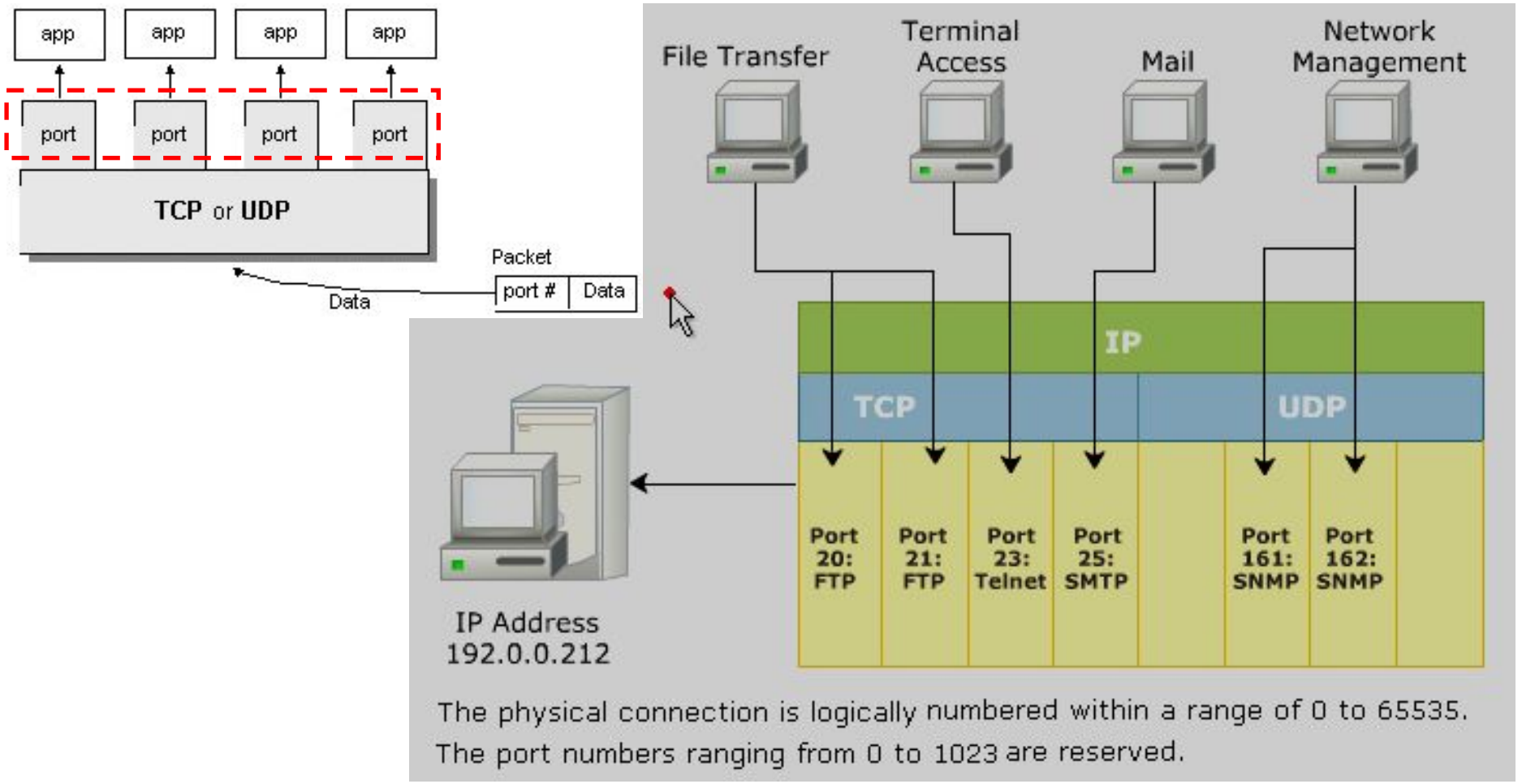
IP:152.3.21.121 or Hostname

Personal computer IP: 127.0.0.1

An IP address is either a 32-bit or 128-bit unsigned number used by IP, a lower-level protocol on which protocols like UDP and TCP are built. The IP address architecture is defined by *RFC 790*

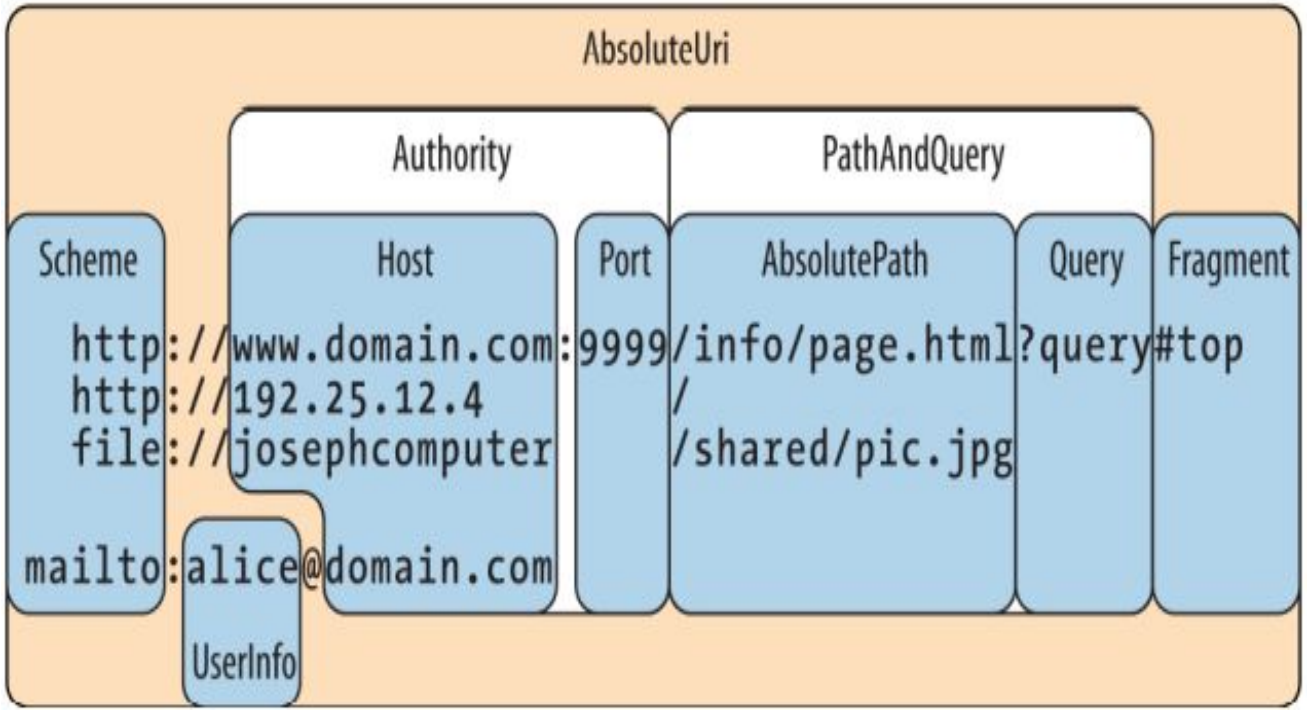
Client-Server Model

- How to distinguish a network-communicating process in a computer?



URL, URN and URI

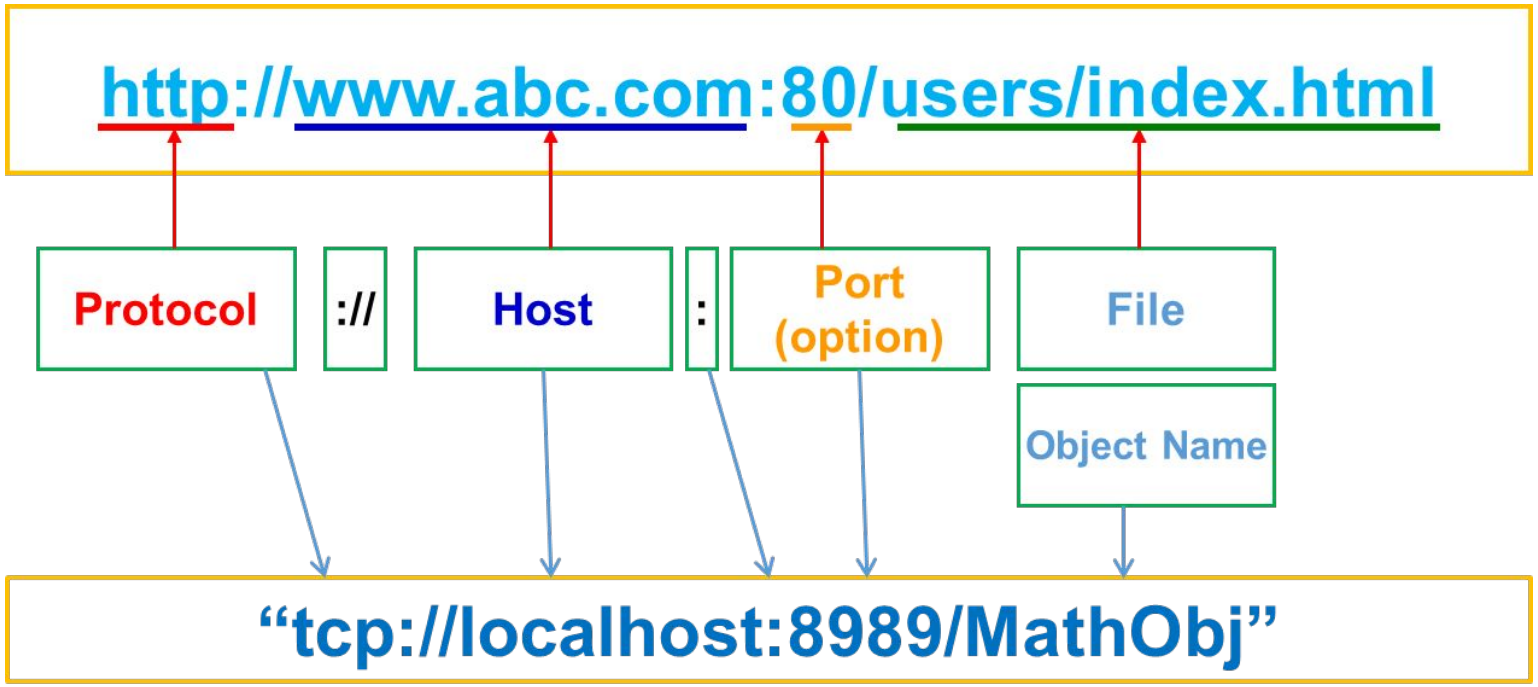
- A URI (Uniform Resource Identifier) is a specially formatted string that describes a resource on the internet or a LAN, such as a web page, file, or email address
- A URI can be broken up into a series of elements—typically, *scheme, authority, and path*
- The Uri class in the System namespace performs just this division, exposing a property for each element



URI properties

URL, URN and URI

- URL stands for Uniform Resource Location. URL is a subset of URI that describes the network address or location where the source is available
- URL begins with the name of the protocol to be used for accessing the resource and then specific resource location



URL, URN and URI

- URLs build on the Domain Name Service (DNS) to address hosts symbolically and use a file-path like syntax to identify specific resources at a given host. For this reason, mapping URLs to physical resources is straightforward and is implemented by various Web browsers
- URN stands for Uniform Resource Name. It is a URI that uses a URN scheme
  - “urn” scheme: It is followed by a namespace identifier, followed by a colon, followed by namespace specific string. For example : urn:isbn:0451450523
  - URN does not imply the availability of the identified resource.URNs are location-independent resource identifiers and are designed to make it easy to map other namespaces into URN space

URIs Demo

```
using static System.Console;
class Program {
    static void Main(string[] args)
    {
        Uri info = new Uri("http://www.domain.com:80/info?id=123#fragment");
        Uri page = new Uri("http://www.domain.com/info/page.html");
        WriteLine($"Host: {info.Host}");
        WriteLine($"Port: {info.Port}");
        WriteLine($"PathAndQuery: {info.PathAndQuery}");
        WriteLine($"Query: {info.Query}");
        WriteLine($"Fragment: {info.Fragment}");
        WriteLine($"Default HTTP port: {page.Port}");
        WriteLine($"IsBaseOf: {info.IsBaseOf(page)}");
        Uri relative = info.MakeRelativeUri(page);
        WriteLine($"IsAbsoluteUri: {relative.IsAbsoluteUri}");
        WriteLine($"RelativeUri: {relative.ToString()}");
        ReadLine();
    }
}
```

```
Host: www.domain.com
Port: 80
PathAndQuery: /info?id=123
Query: ?id=123
Fragment: #fragment
Default HTTP port: 80
IsBaseOf: True
IsAbsoluteUri: False
RelativeUri: info/page.html
```

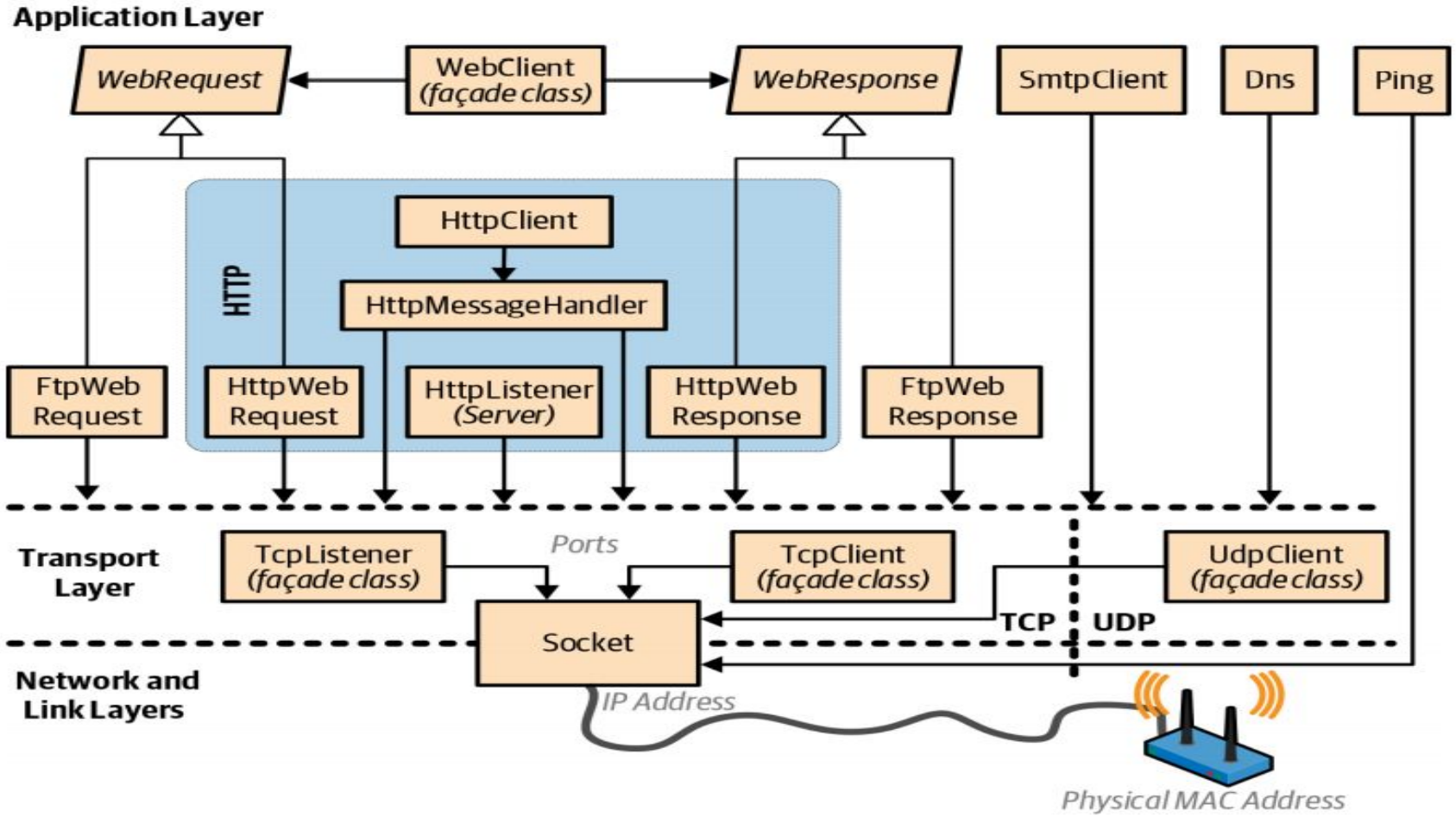
Networking Programming in .NET

Understanding System.Net.* Namespaces

The .NET offers a variety of classes in the System.Net.* namespaces for communicating via standard network protocols, such as HTTP, TCP/IP, and FTP. The summary key components as follows :

- A WebClient facade class for simple download/upload operations via HTTP or FTP
- WebRequest and WebResponse classes for low-level control over client-side HTTP or FTP operations
- HttpClient for consuming HTTP web APIs and RESTful services
- HttpListener for writing an HTTP server
- SmtpClient for constructing and sending mail messages via SMTP
- Dns for converting between domain names and addresses
- TcpClient, UdpClient, TcpListener, and Socket classes for direct access to the transport and network layers

Network Architecture



The figure illustrates .NET networking types and the communication layers in which they reside

Understanding WebRequest Class

- WebRequest and WebResponse are common base classes for managing both HTTP and FTP client-side activity as well as the “file:” protocol. They encapsulate the request/response model that these protocols all share: the client makes a request, and then awaits a response from a server

- WebRequest is the abstract base class for .NET's request/response model for accessing data from the Internet

- An application that uses the request/response model can request data from the Internet in a protocol-agnostic manner, in which the application works with instances of the WebRequest class while protocol-specific descendant classes carry out the details of the request

Understanding WebRequest Class

- The following table describes some of the key properties:

Property Name	Description
ContentLength	When overridden in a descendant class, gets or sets the content length of the request data being sent
ContentType	When overridden in a descendant class, gets or sets the content type of the request data being sent
Credentials	When overridden in a descendant class, gets or sets the network credentials used for authenticating the request with the Internet resource
Method	When overridden in a descendant class, gets or sets the protocol method to use in this request
Headers	When overridden in a descendant class, gets or sets the collection of header name/value pairs associated with the request
RequestUri	When overridden in a descendant class, gets the URI of the Internet resource associated with the request
Timeout	Gets or sets the length of time, in milliseconds, before the request times out

Understanding WebRequest Class

- The following table describes some of the key methods:

Method Name	Description
Create(Uri)	Initializes a new WebRequest instance for the specified URI scheme
GetRequestStream()	When overridden in a descendant class, returns a Stream for writing data to the Internet resource
GetResponse()	When overridden in a descendant class, returns a response to an Internet request
CreateHttp(String)	Initializes a new HttpWebRequest instance for the specified URI string
BeginGetRequestStream(AsyncCallback, Object)	When overridden in a descendant class, provides an asynchronous version of the GetRequestStream() method
BeginGetResponse(AsyncCallback, Object)	When overridden in a descendant class, begins an asynchronous request for an Internet resource
Abort()	Aborts the request

Understanding WebResponse Class

- The WebResponse class is the abstract base class from which protocol-specific response classes are derived
- Applications can participate in request and response transactions in a protocol-agnostic manner using instances of the WebResponse class while protocol-specific classes derived from WebResponse carry out the details of the request
- Client applications do not create WebResponse objects directly, they are created by calling the GetResponse method on a WebRequest instance

Understanding WebResponse Class

- The following table describes some of the key properties and methods:

Property Name	Description
ContentLength	When overridden in a descendant class, gets or sets the content length of data being received
ContentType	When overridden in a derived class, gets or sets the content type of the data being received
Headers	When overridden in a derived class, gets a collection of header name-value pairs associated with this request
IsFromCache	Gets a Boolean value that indicates whether this response was obtained from the cache
Method Name	
Close()	When overridden by a descendant class, closes the response stream
GetResponseStream()	When overridden in a descendant class, returns the data stream from the Internet resource

WebRequest & WebResponse Demo

```
using System;
using System.IO;
using System.Net;
```

```
class Program {
    static void Main(string[] args){
        // Create a request for the URL.
        WebRequest request = WebRequest.Create("http://www.contoso.com/default.html");
        // If required by the server, set the credentials.
        request.Credentials = CredentialCache.DefaultCredentials;
        // Get the response.
        HttpWebResponse response = (HttpWebResponse)request.GetResponse();
        // Display the status.
        Console.WriteLine("Status: " + response.StatusDescription);
        Console.WriteLine(new string('*',50));
        // Get the stream containing content returned by the server.
        Stream dataStream = response.GetResponseStream();
        // Open the stream using a StreamReader for easy access.
        StreamReader reader = new StreamReader(dataStream);
        // Read the content.
        string responseFromServer = reader.ReadToEnd();
        // Display the content.
        Console.WriteLine(responseFromServer);
        Console.WriteLine(new string('*', 50));
        // Cleanup the streams and the response.
        reader.Close();
        dataStream.Close();
        response.Close();
    }
}
```

```
Microsoft Visual Studio Debug...
Status: OK
**************************************************

<!DOCTYPE html>
<html lang="vi-vn" dir="ltr">
<head data-info="{&amp;quot;v&amp;quot;
:&amp;quot;1.0.7797.2686&amp;quot;,&amp;quo
```

Understanding HttpClient Class

- HttpClient provides another layer on top of HttpWebRequest and HttpWeb Response
- HttpClient was written in response to the growth of HTTP-based web APIs and REST services to provide a better experience than WebClient class (WebClient class provides common methods for sending data to or receiving data from any local, intranet, or Internet resource identified by a URI) when dealing with protocols more elaborate than simply fetching a web page
- HttpClient is a newer API for working with HTTP and is designed to work well with web APIs, REST-based services, and custom authentication schemes
- In .NET Framework, HttpClient relied on WebRequest and WebResponse, but in .NET Core, it handles HTTP itself

Understanding HttpClient Class

- An HttpClient instance is a collection of settings applied to all requests executed by that instance. In addition, every HttpClient instance uses its own connection pool, isolating its requests from requests executed by other HttpClient instances
- HttpClient has a richer and extensible type system for headers and content
- HttpClient lets us write and plug in custom message handlers. This enables mocking in unit tests, and the creation of custom pipelines (for logging, compression, encryption, and so on)

Understanding HttpClient Class

- The following table describes some of the key properties and methods:

Property Name	Description
BaseAddress	Gets or sets the base address of Uniform Resource Identifier (URI) of the Internet resource used when sending requests
MaxResponseContentBufferSize	Gets or sets the maximum number of bytes to buffer when reading the response content
Timeout	Gets or sets the timespan to wait before the request times out
Method Name	
GetAsync(String)	Send a GET request to the specified Uri as an asynchronous operation
GetStringAsync(String)	Send a GET request to the specified Uri and return the response body as a string in an asynchronous operation
PostAsync(String, HttpContent)	Send a POST request to the specified Uri as an asynchronous operation
PutAsync(String, HttpContent)	Send a PUT request to the specified Uri as an asynchronous operation
DeleteAsync(String)	Send a DELETE request to the specified Uri as an asynchronous operation

HttpClient Class Demo-01

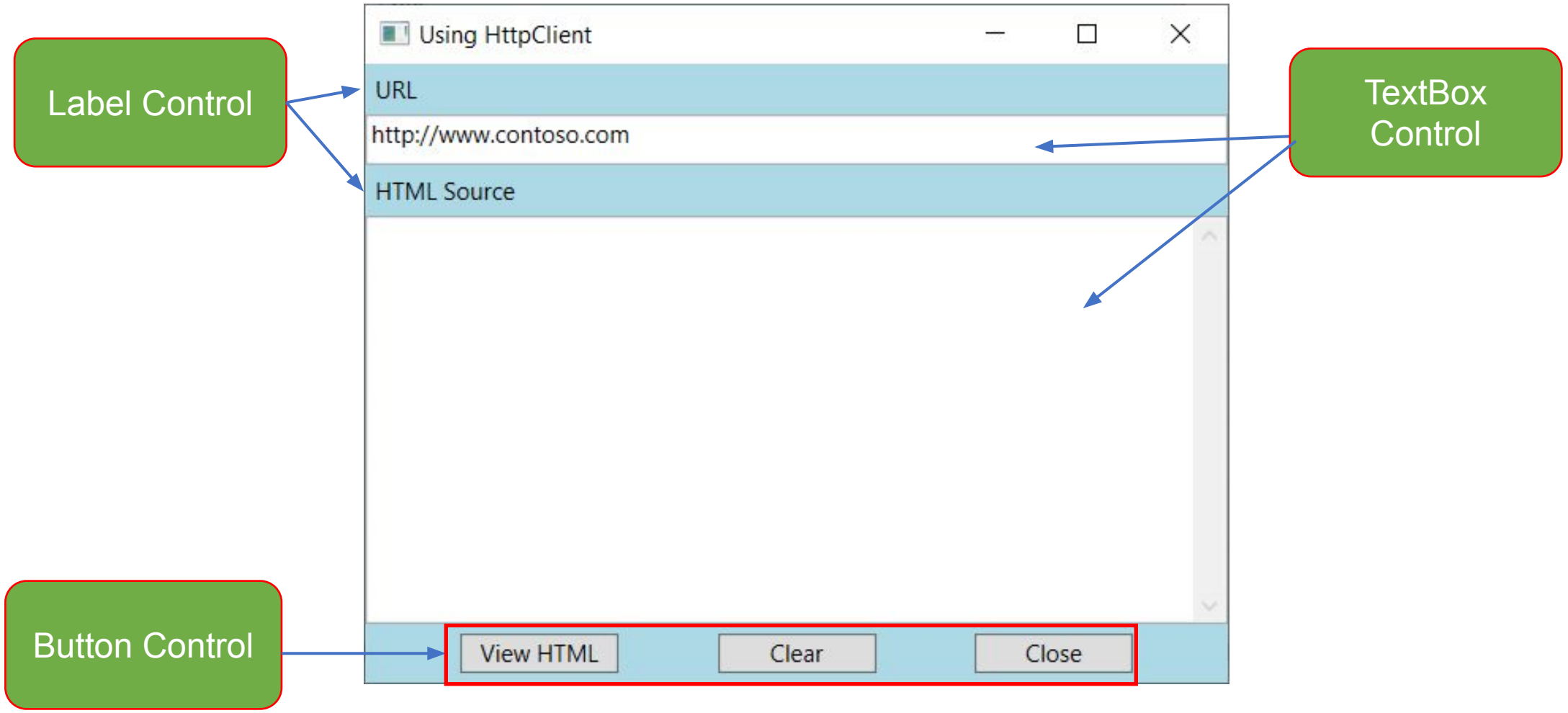
```
using System;
using System.Net.Http;
using System.Threading.Tasks;

class Program {
    // HttpClient is intended to be instantiated once per application
    static readonly HttpClient client = new HttpClient();
    static async Task Main() {
        string uri = "http://www.contoso.com/";
        // Call asynchronous network methods in a try/catch block to handle exceptions.
        try
        {
            HttpResponseMessage response = await client.GetAsync(uri);
            response.EnsureSuccessStatusCode();
            string responseBody = await response.Content.ReadAsStringAsync();
            Console.WriteLine(responseBody);
        }
        catch (HttpRequestException e)
        {
            Console.WriteLine("\nException Caught!");
            Console.WriteLine("Message :{0} ", e.Message);
        }
    }
}
```

Microsoft Visual Studio Debug Console

```
<!DOCTYPE html>
<html lang="vi-vn" dir="ltr">
<head data-info="{&amp;quot;v&amp;quot;:&amp;quot;1.0.7797.2686&amp;quot;,&amp;quot;a&amp;quot;:&amp;quot;46865a80-3b19-4155-b677-ec610a18938b&amp;quot;,&amp;quot;cn&amp;quot;:&amp;quot;OneDeployContainer&amp;quot;,&amp;quot;az&amp;quot;:&amp;quot;{did:92e7dc58ca2143cfb2c818b047cc5cd1, rid: OneDeployContainer, sn: marketingsites-prod-odeastasia, dt: 2018-05-03T20:14:23.4188992Z, bt: 2021-05-07T09:29:32.0000000Z}&amp;quot
```

HttpClient Class Demo-02

1.Create a WPF app named **DemoHttpClient** that has UI as follows :



XAML code of **MainWindow.xaml**:

```
<Window x:Class="DemoHttpClient.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:DemoWPF_HttpClient"
        mc:Ignorable="d"
        Title="Using HttpClient"
        WindowStartupLocation="CenterScreen"
        Width="450" Height="350" >
    <Grid>
    </Grid>
<Window>
```

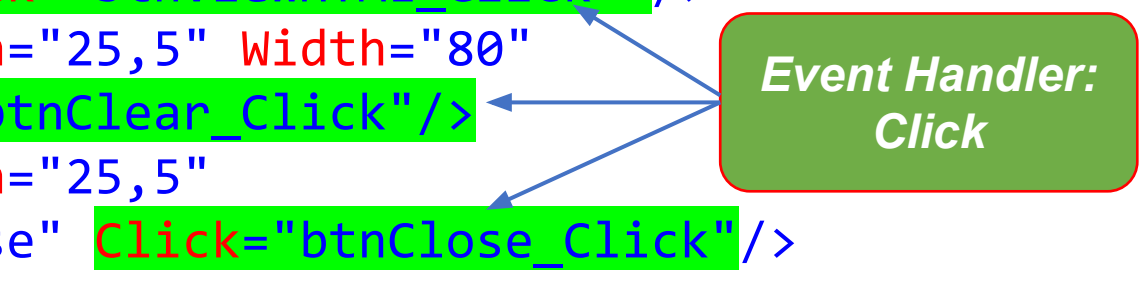
View details in next slide

XAML code of **Grid** tag - **MainWindow.xaml**:

```
<Grid Background="LightBlue"  >
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"></RowDefinition>
            <RowDefinition Height="*"></RowDefinition>
            <RowDefinition Height="Auto"></RowDefinition>
        </Grid.RowDefinitions>
        <StackPanel VerticalAlignment="Stretch"
                    HorizontalAlignment="Stretch"
                    Orientation="Vertical">
            <Label Name="lblURL" Content="URL"/>
            <TextBox Name="txtURL" Text="http://www.contoso.com" Height="25" />
            <Label Name="lbContent"  Content="HTML Source" />
        </StackPanel>
```

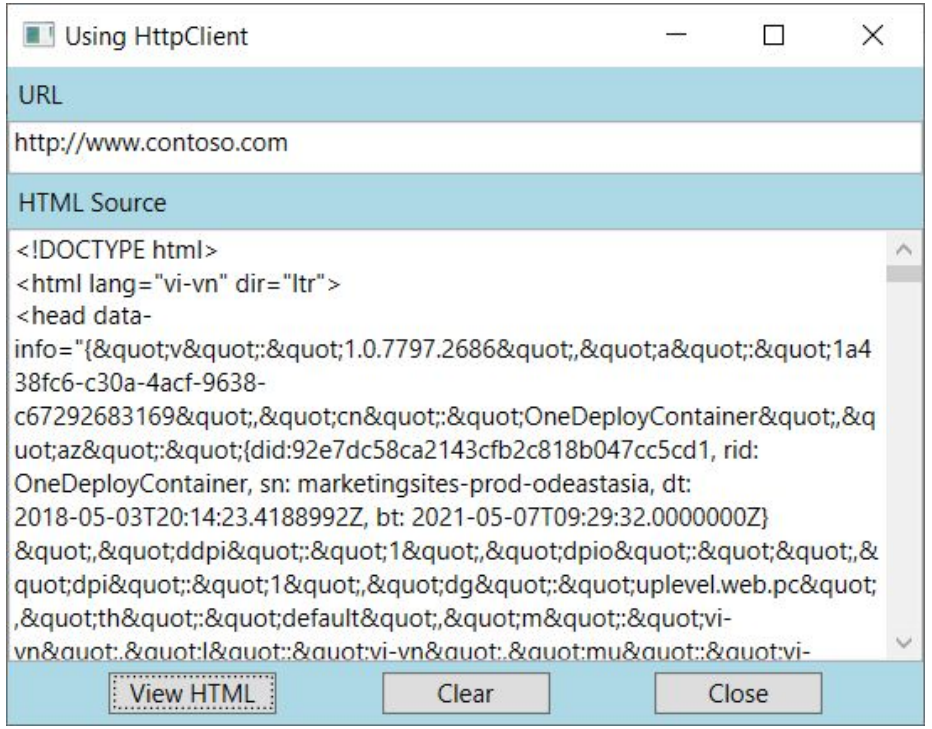
XAML code of **Grid** tag - **MainWindow.xaml**:

```
        <TextBox Grid.Row="1" Name="txtContent"  TextWrapping="Wrap"
                 HorizontalScrollBarVisibility="Auto"
                 AcceptsReturn="True"
                 VerticalScrollBarVisibility="Visible" />
        <StackPanel Grid.Row="2" Orientation="Horizontal"
                 HorizontalAlignment="Center"
                 VerticalAlignment="Top" >
            <Button x:Name="btnViewHTML" Margin="25,5" Width="80"
                 Content="View HTML" Click="btnViewHTML_Click" />
            <Button x:Name="btnClear" Margin="25,5" Width="80"
                 Content="Clear" Click="btnClear_Click"/>
            <Button x:Name="btnClose" Margin="25,5"
                 Width="80" Content="Close" Click="btnClose_Click"/>
        </StackPanel>
</Grid>
```



2.Write codes in **MainWindow.xaml.cs** and run the project as follows:

```
//...
using System.Net.Http;
public partial class MainWindow : Window{
    public MainWindow()...
    readonly HttpClient client = new HttpClient();
    private void btnClose_Click(object sender, RoutedEventArgs e) => Close();
    private void btnClear_Click(object sender, RoutedEventArgs e){
        txtContent.Text = string.Empty;
    }
    private async void btnViewHTML_Click(object sender, RoutedEventArgs e){
        string uri = txtURL.Text;
        // Call asynchronous network methods
        // in a try/catch block to handle exceptions
        try{
            string responseBody = await client.GetStringAsync(uri);
            txtContent.Text = responseBody.Trim();
        }
        catch (HttpRequestException ex){
            MessageBox.Show($"Message :{ex.Message}");
        }
    }
}
```

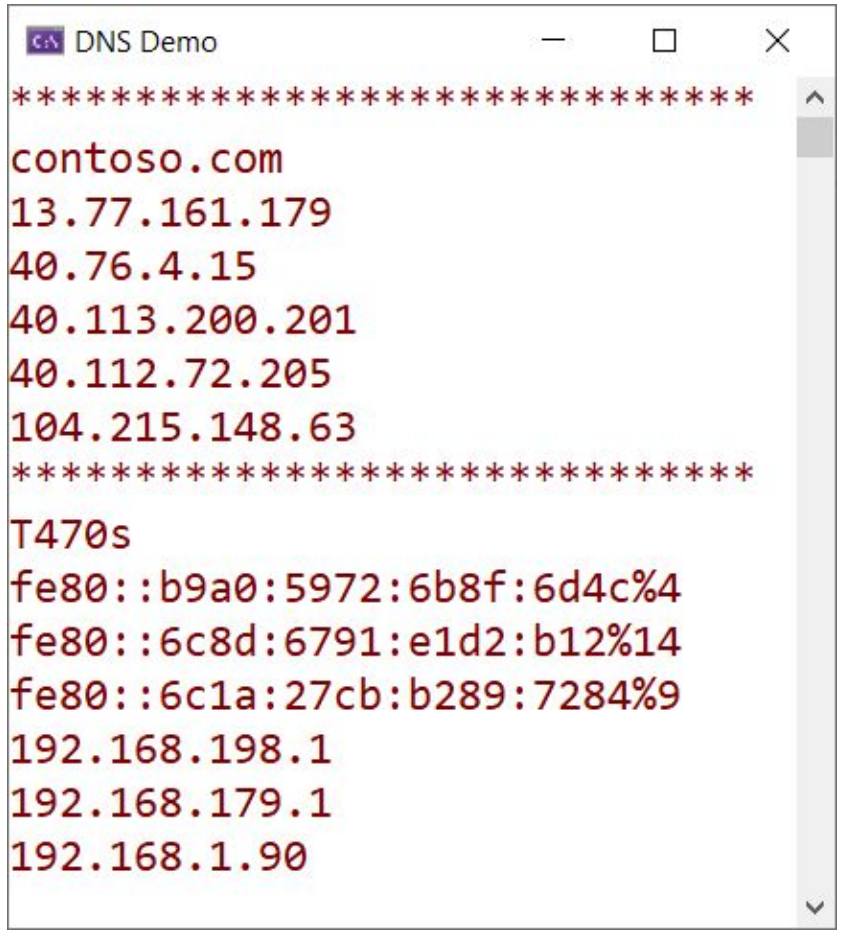


Understanding Domain Name System (DNS)

- Domain Name System (DNS) is the process , which converts Internet address in mnemonic form into the equivalent number IP address
- DNS can also be considered as an database that is present on various computers and has names and IP address of various hosts on the internet
- The DNS consists of three components:
  - The first is a “Name Space” that establishes the syntactical rules for creating and structuring legal DNS names
  - The second is a “Globally Distributed Database” implemented on a network of “Name Servers”
  - The third is "Resolver" software, which understands how to formulate a DNS query and is built into practically every Internet-capable application

Understanding Domain Name System (DNS)

```
using System.Net;
class DnsTestProgram {
    static void Main(string[] args) {
        Console.WriteLine(new string('*',30));
        var domainEntry = Dns.GetHostEntry("www.contoso.com");
        Console.WriteLine(domainEntry.HostName);
        foreach (var ip in domainEntry.AddressList) {
            Console.WriteLine(ip);
        }
        Console.WriteLine(new string('*', 30));
        var domainEntryByAddress = Dns.GetHostEntry("127.0.0.1");
        Console.WriteLine(domainEntryByAddress.HostName);
        foreach (var ip in domainEntryByAddress.AddressList) {
            Console.WriteLine(ip);
        }
        Console.ReadLine();
    }
}
```



The System.Net.Sockets Namespace

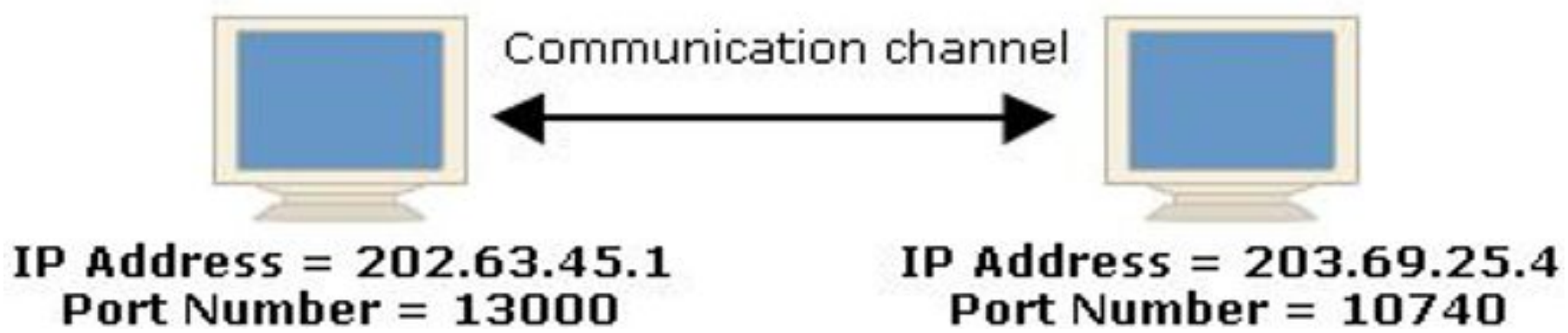
- Provides a managed implementation of the Windows Sockets (Winsock) interface for developers who need to tightly control access to the network
- The following table describes some of the key classes:

Class Name	Description
Socket	Implements the Berkeley sockets interface
TcpClient	Provides client connections for TCP network services
TcpListener	Listens for connections from TCP network clients
UdpClient	Provides User Datagram Protocol (UDP) network services
NetworkStream	Provides the underlying stream of data for network access
SocketAsyncEventArgs	Represents an asynchronous socket operation
SocketException	The exception that is thrown when a socket error occurs
SocketTaskExtensions	This class contains extension methods to the Socket class

Working TCP Services

- The Tranmission Control Protocol (TCP) services contain classes and methods for connecting and sending data between two points or more points. A point consists of both an IP (Internet Protocol) and port number

- The TcpClient and TcpListener classes create the TCP connections on the internet and contain methods and properties for connecting, sending and receiving stream data over the network



The TcpListener Class

- The TcpListener class provides simple methods that listen for and accept incoming connection requests in blocking synchronous mode. We can use either a TcpClient or a Socket to connect with a TcpListener
- The following table describes some of the key methods:

Method Name	Description
AcceptSocket()	Accepts a pending connection request
AcceptSocketAsync()	Accepts a pending connection request as an asynchronous operation
AcceptTcpClient()	Accepts a pending connection request
AcceptTcpClientAsync()	Accepts a pending connection request as an asynchronous operation
Start()	Starts listening for incoming connection requests
Stop()	Closes the listener
Pending()	Determines if there are pending connection requests

The TcpClient Class

- The TcpClient class provides simple methods for connecting, sending, and receiving stream data over a network in synchronous blocking mode
- In order for TcpClient to connect and exchange data, a TcpListener or Socket created with the TCP ProtocolType must be listening for incoming connection requests. We can connect to this listener in one of the following two ways:
  - Create a TcpClient and call one of the three available Connect methods
  - Create a TcpClient using the host name and port number of the remote host. This constructor will automatically attempt a connection

The TcpClient Class

- The following table describes some of the key properties:

Property Name	Description
Active	Gets or sets a value that indicates whether a connection has been made
Available	Gets the amount of data that has been received from the network and is available to be read
Client	Gets or sets the underlying Socket
Connected	Gets a value indicating whether the underlying Socket for a TcpClient is connected to a remote host
ReceiveBufferSize	Gets or sets the size of the receive buffer
ReceiveTimeout	Gets or sets the amount of time a TcpClient will wait to receive data once a read operation is initiated
SendBufferSize	Gets or sets the size of the send buffer
SendTimeout	Gets or sets the amount of time a TcpClient will wait for a send operation to complete successfully
ReceiveBufferSize	Gets or sets the size of the receive buffer

The TcpClient Class

- The following table describes some of the key methods:

Method Name	Description
Connect(IPAddress, Int32)	Connects the client to a remote TCP host using the specified IP address and port number
ConnectAsync(IPAddress, Int32)	Connects the client to a remote TCP host using the specified IP address and port number as an asynchronous operation
BeginConnect(IPAddress, Int32, AsyncCallback, Object)	Begins an asynchronous request for a remote host connection. The remote host is specified by an IPAddress and a port number (Int32)
GetStream()	Returns the NetworkStream used to send and receive data
EndConnect(IAsyncResult)	Ends a pending asynchronous connection attempt
Close()	Disposes this TcpClient instance and requests that the underlying TCP connection be closed
Finalize()	Frees resources used by the TcpClient class
Dispose()	Releases the managed and unmanaged resources used by the TcpClient

Understanding Socket

- Sockets in computer networks are used to establish a connection between two or more computers and used to send data from one computer to another. Each computer in the network is called a node
- A socket is an object that represents a low-level access point to the IP stack. This socket can be open or closed or one of a set number of intermediate states
- Sockets use nodes' IP addresses and a network protocol to create a secure channel of communication and use this channel to transfer data

The Socket Class

- The Socket class provides a rich set of methods and properties for network communications
- The Socket class allows us to perform both synchronous and asynchronous data transfer using any of the communication protocols listed in the ProtocolType enumeration.
- The following table describes some of the key properties and methods:

Property Name	Description
Available	Gets the amount of data that has been received from the network and is available to be read
Connected	Gets a value that indicates whether a Socket is connected to a remote host as of the last Send or Receive operation
Blocking	Gets or sets a value that indicates whether the Socket is in blocking mode.
ReceiveBufferSize	Gets or sets a value that specifies the size of the receive buffer of the Socket
SendTimeout	Gets or sets a value that specifies the amount of time after which a synchronous Send call will time out

The Socket Class

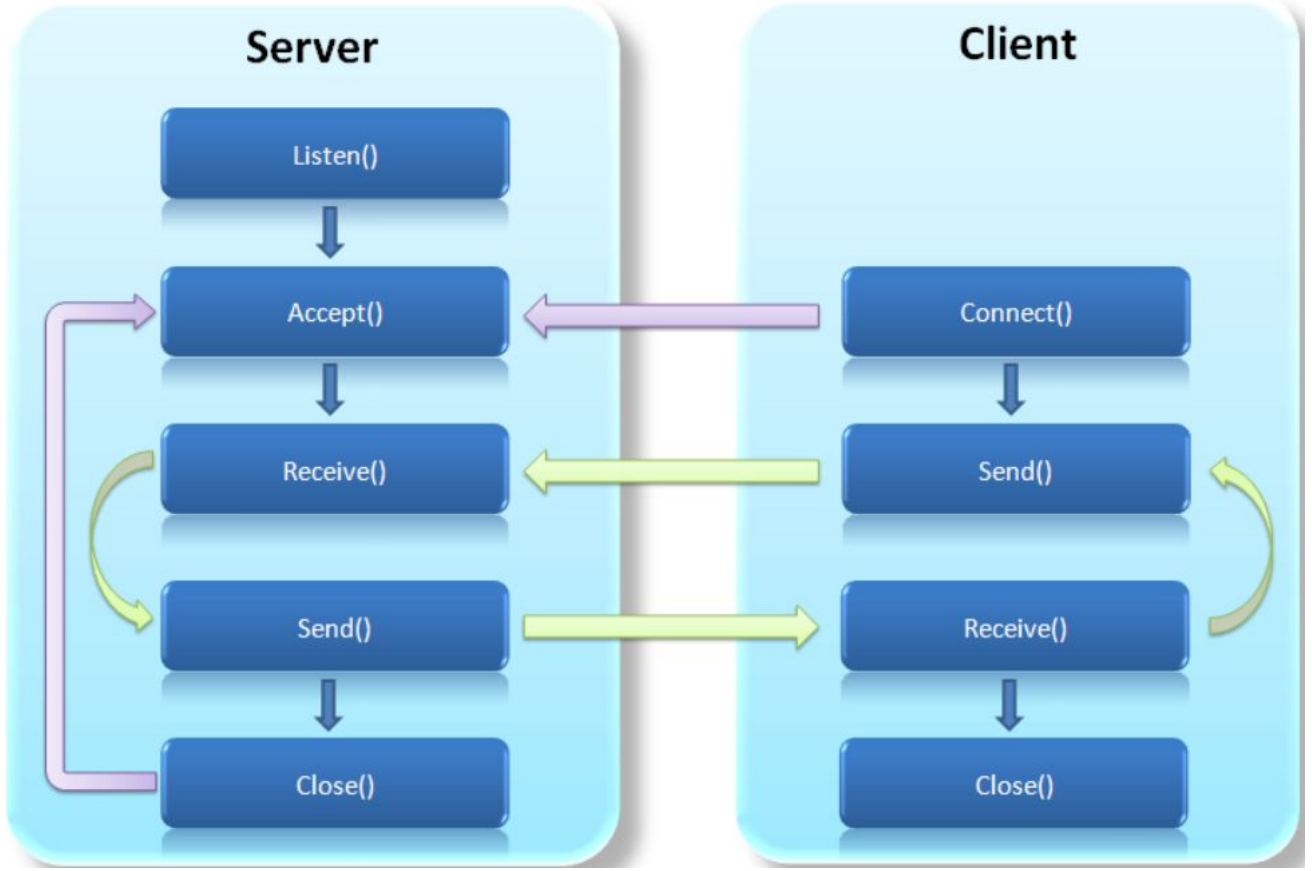
Property Name	Description
ReceiveTimeout	Gets or sets a value that specifies the amount of time after which a synchronous Receive call will time out
SendBufferSize	Gets or sets a value that specifies the size of the send buffer of the Socket

Method Name	Description
Accept()	Creates a new Socket for a newly created connection
Connect(IPAddress, Int32)	Establishes a connection to a remote host. The host is specified by an IP address and a port number
Listen()	Places a Socket in a listening state
SendFile(String)	Sends the file fileName to a connected Socket object with the UseDefaultWorkerThread transmit flag
Send(Byte[])	Sends data to a connected Socket
Receive(Byte[])	Receives data from a bound Socket into a receive buffer
Close()	Closes the Socket connection and releases all associated resources

Using TCP Services Demonstration

How do we develop?

1. Create a Solution named **DemoTCPService**
2. Addition to this solution two Console projects named **ServerApp** and **ClientApp**



3. Write codes in Program.cs of the **ServerApp** as follows :

```
using System;
using System.IO;
using System.Net;
using System.Net.Sockets;
using System.Text;
using System.Threading;

class Program {

    static  void ProcessMessage(object parm)...
    //--------------------------------------------
    static void ExecuteServer(string host, int port)...
    //--------------------------------------------
    public static void Main() {
        string host = "127.0.0.1";
        // Set the TcpListener on port 13000.
        int port = 13000;
        ExecuteServer(host, port);
    }//end Main
}//end Program
```

View details in next slide

Write codes in **ProcessMessage** method as follows :

```
static void ProcessMessage(object parm){
    string data ;
    int count;
    try {
        TcpClient client = parm as TcpClient;
        // Buffer for reading data
        Byte[] bytes = new Byte[256];
        // Get a stream object for reading and writing
        NetworkStream stream = client.GetStream();
        // Loop to receive all the data sent by the client.
        while ((count = stream.Read(bytes, 0, bytes.Length)) != 0) {
            // Translate data bytes to a ASCII string.
            data = System.Text.Encoding.ASCII.GetString(bytes, 0, count);
            Console.WriteLine($"Received: {data} at {DateTime.Now:t}");
            // Process the data sent by the client.
            data = $"{data.ToUpper()}";
            byte[] msg = System.Text.Encoding.ASCII.GetBytes(data);
            // Send back a response.
            stream.Write(msg, 0, msg.Length);
            Console.WriteLine($"Sent: {data}");
        }
        // Shutdown and end connection
        client.Close();
    }
    catch (Exception ex) {
        Console.WriteLine("{0}", ex.Message);
        Console.WriteLine("Waiting message...");
    }
}//end ProcessMessage
```

Write codes in **ExecuteServer** method as follows :

```
static void ExecuteServer(string host, int port) {
    int Count = 0;
    TcpListener server = null;
    try {
        Console.Title = "Server Application";
        IPAddress localAddr = IPAddress.Parse(host);
        server = new TcpListener(localAddr, port);
        // Start listening for client requests.
        server.Start();
        Console.WriteLine(new string('*', 40));
        Console.WriteLine("Waiting for a connection... ");
        // Enter the listening loop.
        while (true){
            // Perform a blocking call to accept requests.
            TcpClient client = server.AcceptTcpClient();
            Console.WriteLine($"Number of client connected:{++Count}");
            Console.WriteLine(new string('*', 40));
            //Create a thread to receive and send message
            Thread thread = new Thread(new ParameterizedThreadStart(ProcessMessage));
            thread.Start(client);
        }
    }
    catch (Exception ex) {
        Console.WriteLine("Exception: {0}", ex.Message);
    }
    finally {
        server.Stop();
        Console.WriteLine("Server stopped.  Press any key to exit !");
    }
    Console.Read();
}
```

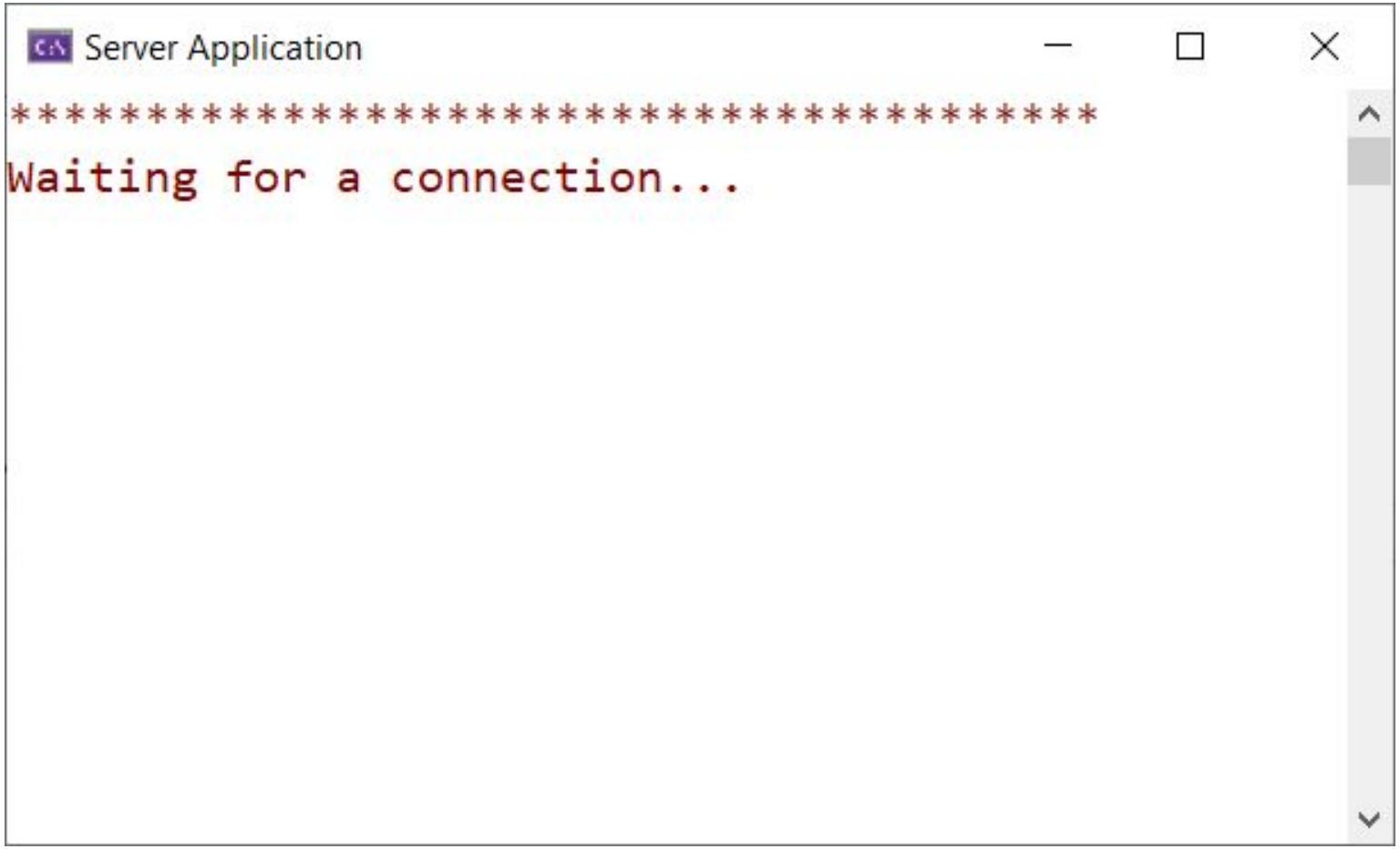
4. Write codes in **Program.cs** of the **ClientApp** as follows :

```
using System;
using System.IO;
using System.Net;
using System.Net.Sockets;
using System.Text;
```

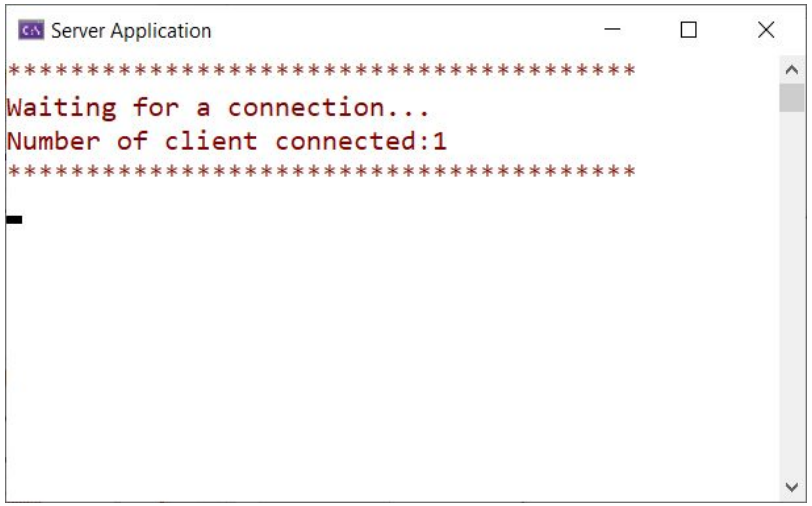
```
class Program {
    static void ConnectServer(String server, int port) {
        string message, responseData;
        int bytes;
        try {
            // Create a TcpClient
            TcpClient client = new TcpClient(server, port);
            Console.Title = "Client Application";
            NetworkStream stream = null;
            while (true) {
                Console.Write("Input message <press Enter to exit>:");
                message = Console.ReadLine();
                if (message == string.Empty)  {
                    break;
                }
                // Translate the passed message into ASCII and store it as a byte array.
                Byte[] data = System.Text.Encoding.ASCII.GetBytes($"{message}");
                // Get a client stream for reading and writing.
                stream = client.GetStream();
                // Send the message to the connected TcpServer.
                stream.Write(data, 0, data.Length);
                Console.WriteLine("Sent: {0}", message);
```

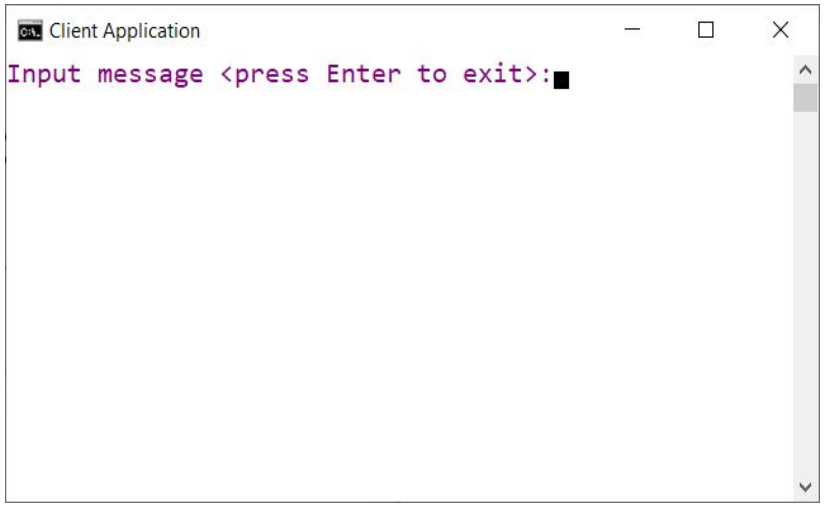
```
                // Receive the TcpServer response.
                // Use buffer to store the response bytes.
                data = new Byte[256];
                // Read the first batch of the TcpServer response bytes.
                bytes = stream.Read(data, 0, data.Length);
                responseData = System.Text.Encoding.ASCII.GetString(data, 0, bytes);
                Console.WriteLine("Received: {0}", responseData);
            }
            // Shutdown and end connection
            client.Close();
        }
        catch (Exception e)  {
            Console.WriteLine("Exception: {0}", e.Message);
        }
    }//end ConnectServer
    //-----------------------------------------------
    static void Main(string[] args) {
        string server = "127.0.0.1";
        // Set the TcpListener on port 13000.
        int port = 13000;
        ConnectServer(server, port);
    }//end Main
}//end Program
```

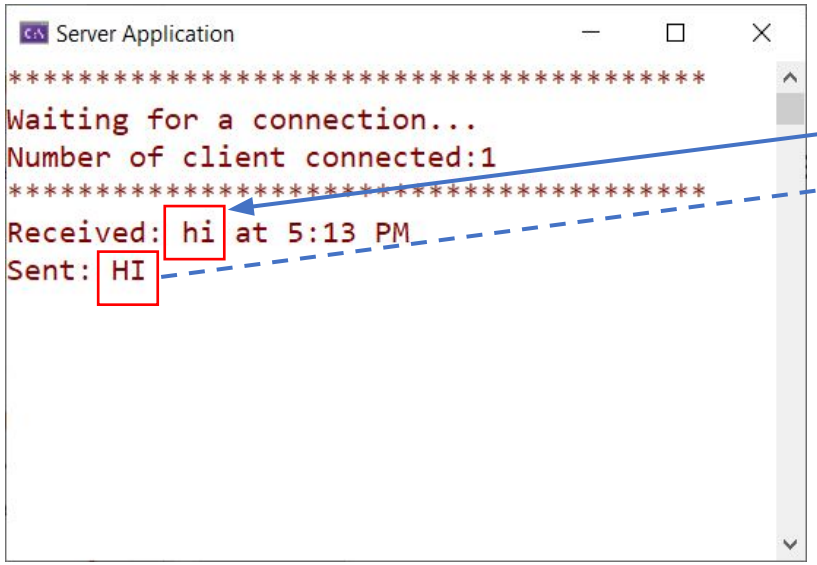
5.Right-click on the **ServerApp** project, select **Set as Startup Project** then press **Ctrl+F5** to run it

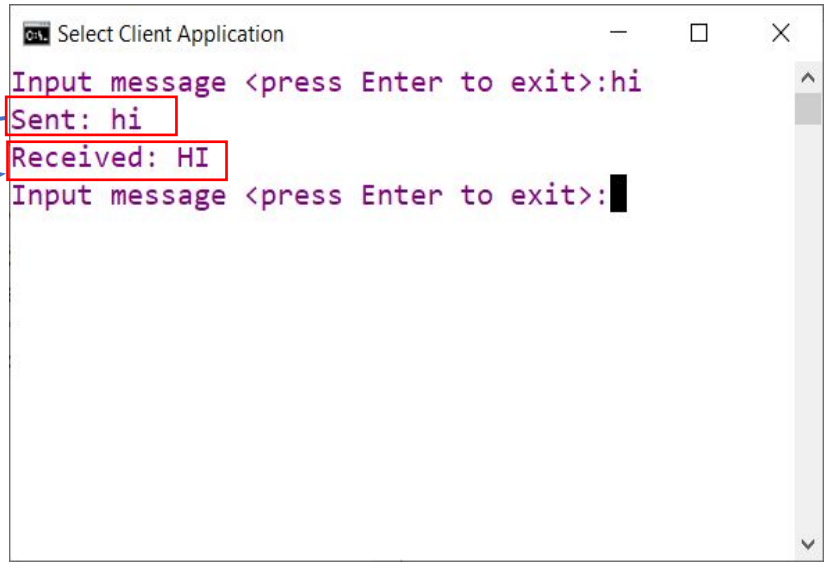


6. Right-click on the **ClientApp** project, select **Set as Startup Project** then press Ctrl+F5 to run it









Working UDP Services

- User Datagram Protocol (UDP) is a simple protocol that makes a best effort to deliver data to a remote host

- The UDP protocol is connectionless protocol thus UDP datagrams sent to the remote endpoint are not guaranteed to arrive and they aren't guaranteed to arrive in the same sequence in which they are sent. Applications that use UDP must be prepared to handle missing, duplicate, and out-of-sequence datagrams

- The **UdpClient** class communicates with network services using UDP. The properties and methods of the UdpClient class abstract the details of creating a **Socket** for requesting and receiving data using UDP

UdpClient Class

- The following table describes some of the key properties and methods:

Property Name	Description
Active	Gets or sets a value indicating whether a default remote host has been established
Available	Gets the amount of data received from the network that is available to read
Client	Gets or sets the underlying network Socket

Method Name	Description
Connect(String, Int32)	Establishes a default remote host using the specified host name and port number
Close()	Closes the UDP connection
Send(Byte[], Int32)	Sends a UDP datagram to a remote host
Receive(IPEndPoint)	Returns a UDP datagram that was sent by a remote host
JoinMulticastGroup(Int32, IPAddress)	Adds a UdpClient to a multicast group
Dispose()	Releases the managed and unmanaged resources used by the UdpClient

Using UDP Services Demonstration

1. Create a Console project named **UDPServerApp** then write codes in Program.cs as follows :

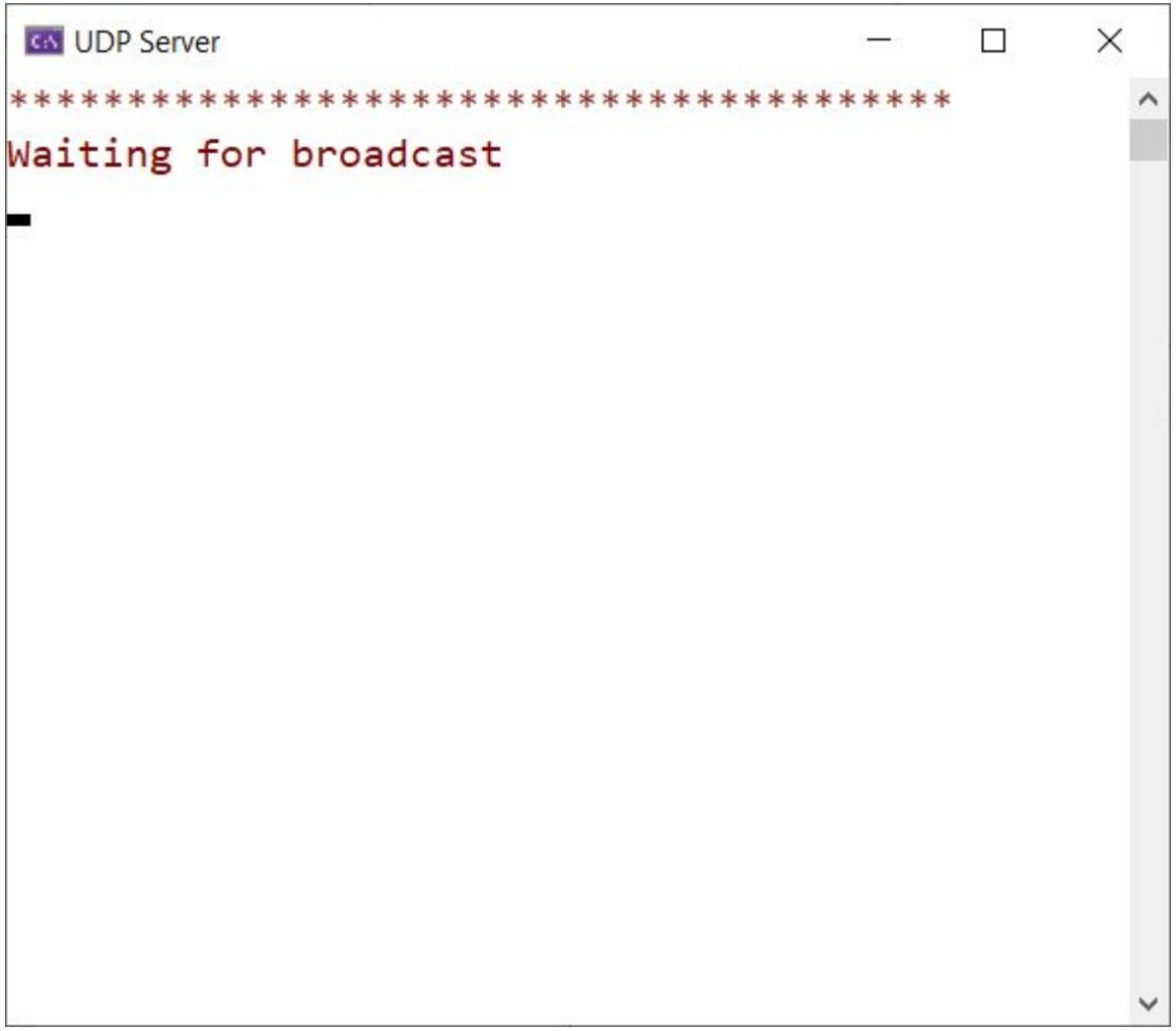
```
using System;
using System.Net;
using System.Net.Sockets;
using System.Text;
using System.Threading;
class Program {
    const int listenPort = 11000;
    const string host = "127.0.0.1";
    private static void StartListener() {
        string message;
        UdpClient listener = new UdpClient(listenPort);
        IPAddress address = IPAddress.Parse(host);
        IPEndPoint remoteEndpoint = new IPEndPoint(address, listenPort);
        Console.Title = "UDP Server";
        Console.WriteLine(new string('*', 40));
        try
        {
            while (true){
                Console.WriteLine("Waiting for broadcast");
                byte[] bytes = listener.Receive(ref remoteEndpoint);
                message = Encoding.ASCII.GetString(bytes, 0, bytes.Length);
                Console.WriteLine($"Received broadcast from {remoteEndpoint }:{message}");
            }
            catch (SocketException e){
                Console.WriteLine(e.Message);
            }
            finally{
                listener.Close();
            }
        }//end StartListener
        //-------------------------------------
        public static void Main() {
            Thread thread = new Thread(
                new ThreadStart(StartListener));
            thread.Start();
        }//end Main
}//end Program
```

2. Create a Console project named **UDPClientApp** then write codes in Program.cs as follows :

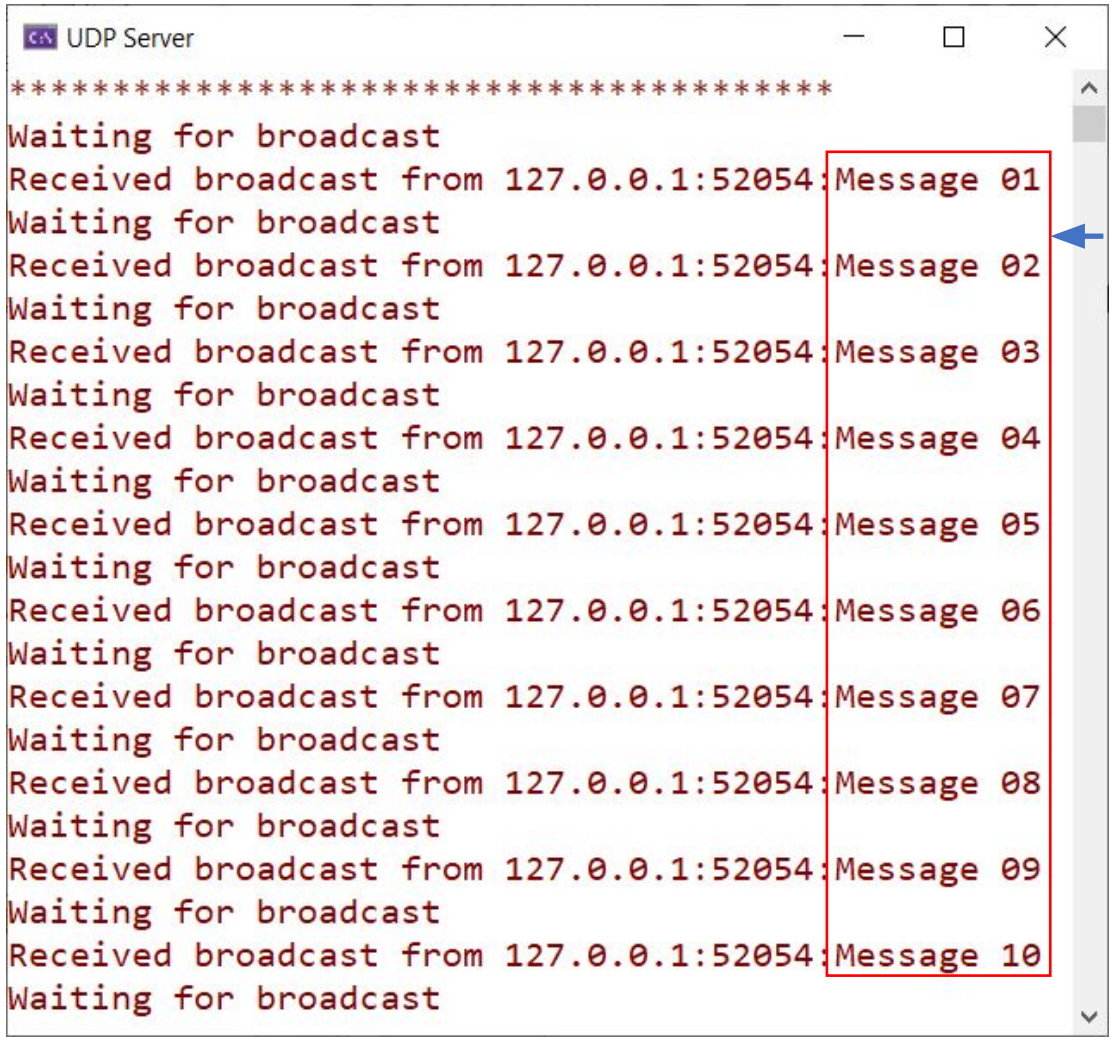
```
using System;
using System.Net;
using System.Net.Sockets;
using System.Text;
using System.Threading;
```

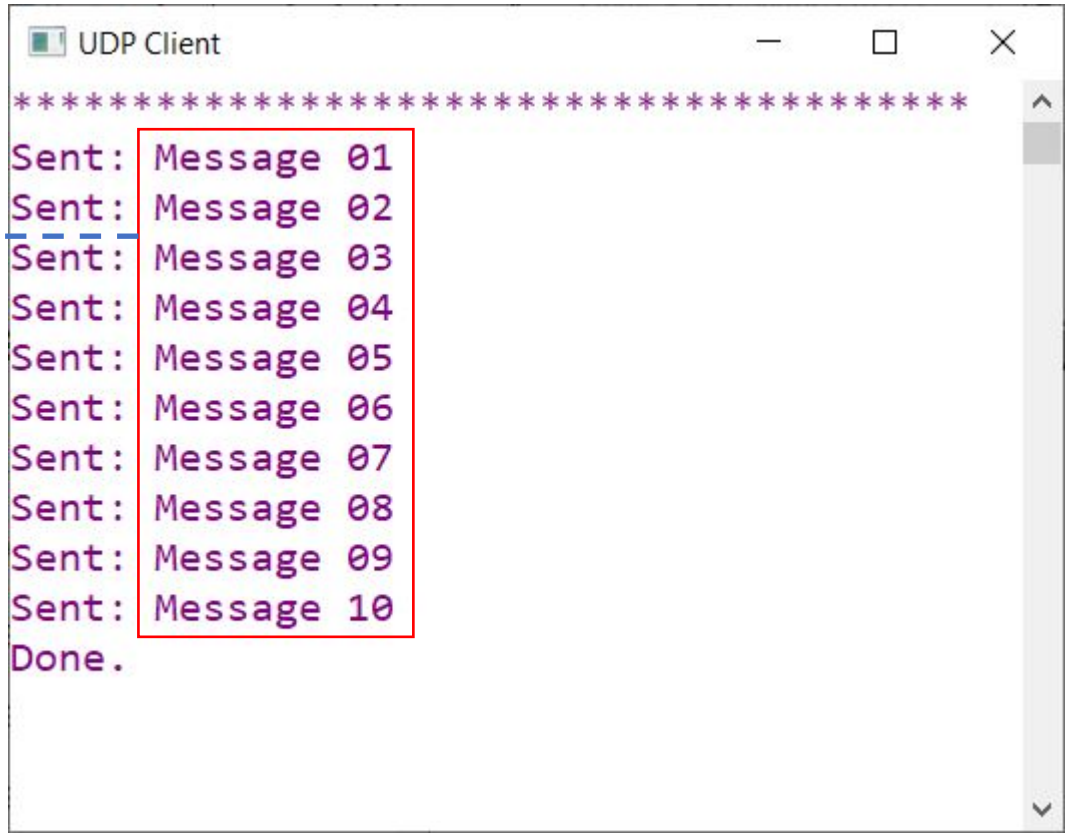
```
class Program{
    static void ConnectServer(string host, int port) {
        UdpClient client = new UdpClient();
        IPAddress address = IPAddress.Parse(host);
        IPEndPoint remoteEndpoint = new IPEndPoint(address, port);
        string message;
        int count = 0;
        bool done = false;
        Console.Title = "UDP Client";
        try{
            Console.WriteLine(new string('*',40));
            client.Connect(remoteEndpoint);
            while (!done){
                message = $"Message {++count:D2}";
                byte[] sendBytes = Encoding.ASCII.GetBytes(message);
                client.Send(sendBytes, sendBytes.Length);
                Console.WriteLine($"Sent: {message}");
                Thread.Sleep(2000);
                if (count == 10)  //broadcast 10 messages
                {
                    done = true;
                    Console.WriteLine("Done.");
                }
            }
        }
        catch (SocketException e) {
            Console.WriteLine(e.Message);
        }
        finally{
            client.Close();
        }
    }//end ConnectServer
    //-------------------------------------------------
    static void Main(string[] args){
        string host = "127.0.0.1";
        int port = 11000;
        ConnectServer(host, port);
        Console.Read();
    }//end Main
}//end Program
```

3. Right-click on the **UDPServerApp** project, select **Set as Startup Project** then press **Ctrl+F5** to run it



4. Right-click on the **UDPClientApp** project, select **Set as Startup Project** then press **Ctrl+F5** to run it





Summary

- Concepts were introduced:
  - Overview Networking Basic
  - Overview Client-Server Model
  - Explain about URL, URN and URI
  - Explain about WebRequest and WebResponse class
  - Explain about HttpClient class
  - Explain about Domain Name System (DNS)
  - Explain about UDP service
  - Overview TCP Services: TcpListener, TcpClient and Socket class
  - Demo WebRequest and HttpClient with .NET application
  - Demo TcpListener and TcpClient with .NET application
  - Demo UDP service with .NET application